

Crafty Controller Wake Proxy

Microservicio proxy TCP inteligente 'Wake-on-Ping' para Minecraft
Optimizacion de recursos en hosts 24/7 y servidores NAS

Detalles del Documento

Proyecto:	Crafty Controller Wake Proxy
Lenguaje:	Python 3.11+ / Dockerizado
Licencia:	MIT (Codigo Abierto)
Panel Compatible:	Crafty Controller v4
Fecha de Generacion:	Agosto 2026
Autor del Proyecto:	Jonatan / Comunidad Open Source

Introduccion

El presente manual documenta el funcionamiento, configuracion y despliegue del microservicio Crafty Controller Wake Proxy. Este sistema permite optimizar drasticamente los recursos del sistema (CPU y memoria RAM) al apagar automaticamente los servidores de Minecraft inactivos y encenderlos bajo demanda de forma transparente en cuanto un jugador intenta conectarse. El microservicio esta completamente dockerizado y disenado para ejecutarse de forma robusta y desatendida en cualquier sistema Linux, Windows o servidores NAS (como Synology DSM).

Tabla de Contenidos

1. Características y Funcionamiento Detallado	Pag. 2
2. Sistemas de Seguridad y Protección Avanzada	Pag. 3
3. Esquema de Red y Puertos	Pag. 4
4. Obtención de Credenciales en Crafty 4	Pag. 5
5. Código Fuente Completo (Dockerfile, docker-compose.yml)	Pag. 6
6. Archivo Compose y Despliegue (Linux y Synology NAS)	Pag. 7
7. Gestión y Buenas Prácticas	Pag. 8
8. Licencia MIT	Pag. 8

1. Características y Funcionamiento Detallado

El microservicio es un proxy inverso TCP transparente que se coloca de forma intermedia entre el cliente de Minecraft y el servidor de juego real gestionado por Crafty Controller v4.

a) Wake-on-Ping con Filtro de Handshake:

El proxy permanece escuchando en el puerto externo asignado. Cuando un cliente intenta conectarse, el proxy lee los primeros bytes del paquete y analiza el protocolo de Handshake de Minecraft. Si detecta que es un simple ping de la lista de servidores (`next_state = 1`) o un escaneo de red, el proxy cierra el socket inmediatamente y no enciende el servidor. Si el jugador hace clic activamente en 'Entrar al servidor' (`next_state = 2`), el proxy intercepta la conexión, consulta a la API de Crafty Controller (`GET /api/v2/servers/{serverID}/stats`), y si está apagado, realiza una petición HTTP POST autorizada con un JWT Token para arrancar el servidor.

El proxy retiene la conexión del jugador e inicia un bucle de verificación asíncrono, consultando el estado de Crafty cada 2 segundos. En el momento en que se confirma que el servidor está activo (ONLINE), crea sockets de red cruzados y reenvía todo el tráfico de juego en tiempo real (enviando primero el handshake inicial).

b) Auto-Sleep con Doble Verificación:

Para optimizar recursos de hardware en el host de destino, un hilo secundario monitoriza permanentemente el socket de red. Para evitar falsos positivos, los pings de actualización de lista de servidores y los escaneos de puertos no reinician el temporizador de inactividad, el cual solo comienza a contar cuando el último jugador real sale del juego.

Además, el proxy detecta cuando el servidor pasa de OFFLINE a ONLINE (ya sea encendido manualmente desde el panel de Crafty o de forma automática por el proxy) y resetea el temporizador de inactividad en ese momento, garantizando siempre el periodo de gracia completo configurado en `IDLE_TIMEOUT_SECONDS` antes de poder apagarlo. Una vez expirado ese tiempo, el proxy realiza una doble comprobación: consulta la API REST de Crafty para obtener el recuento real de jugadores activos en el servidor. El microservicio solo enviará la orden de apagado si no hay conexiones TCP activas en el proxy y la API de Crafty confirma que el recuento de jugadores online es 0.

2. Sistemas de Seguridad y Proteccion Avanzada

El microservicio incorpora mecanismos de proteccion especificos disenados para garantizar la eficiencia energetica y la optimizacion del host, evitando encendidos accidentales y apagados involuntarios:

a) Filtro Inteligente de Handshake (Minecraft Handshake Parser):

Este filtro analiza los primeros bytes de todas las conexiones TCP entrantes al puerto publico del proxy, decodificando en tiempo real el protocolo de Minecraft:

- Si se detecta un Status Ping (`next_state = 1`), el cliente solo esta consultando el estado (ej. refrescando la lista de servidores multijugador o escaneando puertos). Si el servidor esta apagado, el proxy descarta y cierra la conexion de inmediato, evitando arranques accidentales.
- Si se detecta un Login Request (`next_state = 2`), significa que el jugador pulso 'Entrar al servidor'. Solo en este caso el proxy envia la orden de arranque a la API de Crafty.

b) Doble Verificacion de Apagado por API (Inactivity Double Check):

Este sistema actua como un doble escudo de confirmacion antes de apagar la instancia:

- Arranque manual respetado: El proxy monitoriza continuamente el estado del servidor. En cuanto detecta que el servidor pasa de OFFLINE a ONLINE (ya sea encendido manualmente desde el panel de Crafty o por el proxy), resetea automaticamente el temporizador de inactividad. Esto garantiza siempre el periodo de gracia completo de `IDLE_TIMEOUT_SECONDS` (configurado en el `.env`) desde el momento del arranque, evitando apagados inmediatos tras un encendido manual.
- Cuando el temporizador expira con 0 conexiones TCP activas, el proxy consulta la API de Crafty (`GET /api/v2/servers/{serverID}/stats`) y verifica el recuento real de jugadores en linea ('online'). El servidor solo se apagara si el proxy registra 0 conexiones TCP y la API confirma que el recuento es 0.
- Si hay jugadores activos reportados por la API, se reinicia el temporizador, evitando apagados accidentales.

3. Esquema de Red y Puertos

- Puerto Publico / Proxy (LISTEN_PORT):

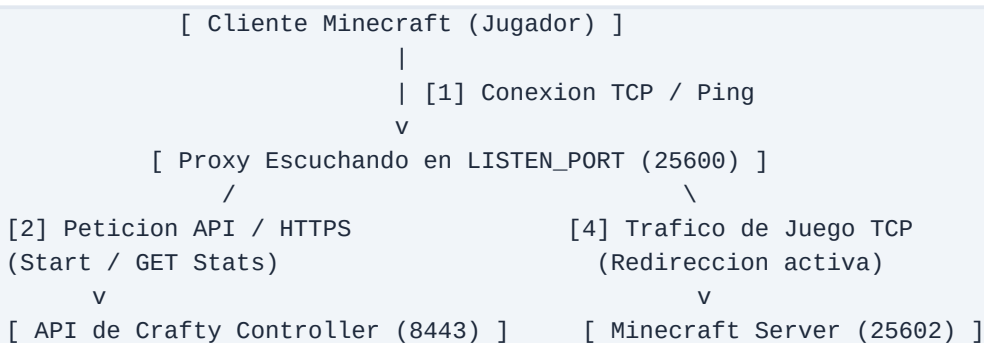
Es el puerto expuesto en internet en tu enrutador (Port Forwarding). Por defecto suele ser el 25565 o 25600. Es el unico puerto que los jugadores remotos necesitan conocer.

- Puerto Interno de Minecraft (INTERNAL_PORT):

Es el puerto que Crafty asigna directamente a la instancia de Minecraft (ej. 25602). Este puerto debe quedar oculto y bloqueado de cara a internet, siendo accesible unicamente de forma interna por el proxy TCP.

- Direccion URL de Crafty (CRAFTY_URL):

El panel de gestion web HTTPS en el puerto por defecto 8443. El contenedor de docker requiere acceso por subred local para consultar y arrancar/detener la instancia. El proxy ignora las alertas de certificados autofirmados mediante deshabilitacion de warnings por software.



4. Obtencion de Credenciales en Crafty 4

A) Generar Token de la API de Crafty (API_TOKEN)

1. Inicia sesion en el panel web de Crafty Controller v4 como Administrador.
2. Haz clic en tu icono de usuario (arriba a la derecha) y selecciona 'Ajustes de la cuenta' (Account Settings).
3. Dirigete a la seccion de 'Claves API' (API Keys).
4. En el campo 'Nombre - ¿Como te gustaria llamar a este Token de API?', introduce un nombre (ej. 'WakeProxy').
5. En la lista de permisos (COMMANDS, TERMINAL, LOGS, etc.), marca la casilla de 'Acceso completo'.
6. Clicka en el boton verde '+Create'.
7. Una vez creado, en la tabla superior haz clic en 'Conseguir un token' (Get a token).
8. **IMPORTANTE:** Copia el token y utilízalo. No vuelvas a hacer clic en este boton. Si haces clic de nuevo, Crafty generara un token nuevo y anulara el anterior.

B) Identificar el UUID del Servidor (SERVER_ID)

1. En el menu principal de Crafty, ve a la pestaña de 'Servers' (Servidores).
2. Haz clic sobre el servidor Minecraft especifico que quieres poner bajo la gestion del proxy.
3. Dirigete a la barra de direcciones de tu navegador web.
4. Copia el UUID alfanumerico completo que se encuentra despues del parametro '?id='.

Por ejemplo, en la direccion URL:

https://panel.crafty.local:8443/panel/server_detail?id=516a6a24-9b36-419b-a010-85f8c6b753a9

El UUID a copiar es: 516a6a24-9b36-419b-a010-85f8c6b753a9

5. Código Fuente Completo

Archivo de Configuración (.env)

Modifica las variables en el archivo .env (creado a partir de .env.example). IMPORTANTE: NO modifiques el archivo de código fuente src/proxy.py, toda la configuración debe hacerse exclusivamente en el archivo .env:

```
CRAFTY_URL=https://192.168.1.100:8443
API_TOKEN=tu_crafty_api_jwt_token_here
SERVER_ID=tu_minecraft_server_uuid_here
TARGET_HOST=127.0.0.1
INTERNAL_PORT=25602
LISTEN_PORT=25600
IDLE_TIMEOUT_SECONDS=600
```

Dockerfile

Entorno de empaquetado Docker basado en Python slim:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/proxy.py .
CMD ["python", "-u", "proxy.py"]
```

6. Compose File y Despliegue

docker-compose.yml

```
version: '3.8'

services:
  crafty_controller-wake_proxy:
    build: .
    container_name: crafty_controller-wake_proxy
    restart: unless-stopped
    network_mode: bridge
    ports:
      - "${LISTEN_PORT:-25600}:${LISTEN_PORT:-25600}"
    env_file:
      - .env
    volumes:
      - ./src/proxy.py:/app/proxy.py
      - .env:/app/.env
```

Guías de Instalación

Método Linux CLI:

1. Crea la carpeta de trabajo: `mkdir -p "/opt/Crafty Controller-Wake Proxy" && cd "/opt/Crafty Controller-Wake Proxy"`
2. Crea la carpeta `src/` y mete `proxy.py` dentro. Crea `Dockerfile`, `docker-compose.yml`, `requirements.txt`, `.gitignore` y `.env` (copiado de `.env.example`).
3. Configura tus datos reales en el archivo `.env`.
4. Ejecuta: `docker compose up -d --build`

Método Synology NAS DSM:

1. Crea una carpeta compartida en File Station (ej. `"/volume1/docker/Crafty Controller-Wake Proxy"`) y sube la estructura del proyecto (carpeta `src`, `Dockerfile`, etc.).
2. Crea un archivo `.env` en esa misma carpeta con tus credenciales locales.
3. Abre Container Manager, ve a Proyectos y haz clic en Crear.
4. Selecciona la carpeta compartida, elige 'Crear `docker-compose.yml`' y pega el código de compose.
5. Finaliza el asistente compilando e iniciando el contenedor. El compose cargará el `.env` y mapeará `./src/proxy.py` automáticamente.

7. Gestion y Buenas Practicas

Comandos de Administracion

- Logs del contenedor en tiempo real: `docker logs -f crafty_controller-wake_proxy`
- Reiniciar contenedor para aplicar cambios en `proxy.py` sin compilar: `docker restart crafty_controller-wake_proxy`
- Detener microservicio temporalmente: `docker compose down`
- Recompilar dependencias o reconstruir imagen: `docker compose up -d --build`

Buenas Practicas y Consejos

- Tiempo de Inactividad: Mantener `IDLE_TIMEOUT_SECONDS` entre 600 (10 mins) y 1800 (30 mins). Evitar valores menores a 5 mins para evitar apagados durante desconexiones accidentales.
- Seguridad: Mantener el API Token protegido. No subirlo a repositorios publicos. Se aconseja usar variables de entorno en sistemas de produccion compartidos o publicos.
- Enrutamiento: Asegurar que el contenedor Docker tenga permisos en red para contactar con la IP/puerto interno del juego y el puerto HTTPS de la API de Crafty Controller.

8. Licencia MIT

Copyright (c) 2026 Jonatan

Este codigo y documentacion se distribuyen bajo los terminos de la Licencia MIT. Queda autorizado el uso, modificacion, distribucion y explotacion comercial sin restricciones, con la unica condicion de incluir el aviso de copyright original en los trabajos derivados.